Thamill Indian B U
Cohort 15

Day 03 Post-Session Assignment & Engineering Reflection

AI Engineering Program | Building Reliable LLM Applications

Part 1 - Challenge Output / Build Evidence

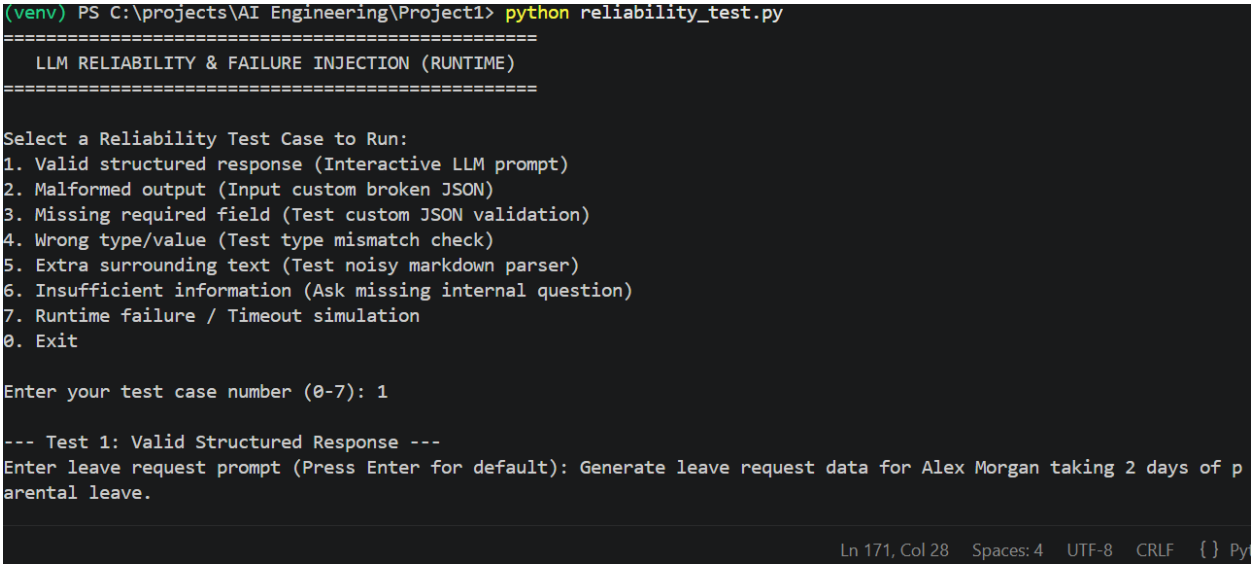
Artifact & Repository: reliability_test.py, built using Python with the OpenAI SDK connected to our local Ollama endpoint running llama3.

Output Contract / Schema: A clean JSON schema requiring employee_name (string), request_type (string), days (integer), and approved (boolean).

Parsing and Validation Logic: We use json.loads() combined with lightweight markdown block stripping to extract the payload, followed by explicit type checks and required key validation.

Error / Fallback Behavior: Robust try-except blocks catch JSONDecodeError and TimeoutError, routing failures to safe fallback messaging instead of crashing.

Retry Configuration: We set up a bounded retry loop targeting transient network drops (TimeoutError) capped at 3 attempts with exponential backoff.



Thamill Indian B U
Cohort 15

Part 2 - Reliability Test Results

Case	Expected Behavior	Actual Result	Pass/Needs Work	Evidence/Notes
Valid structured response	Response passes parsing and validation.	Successfully parsed valid employee JSON schema without errors.	Pass	Loaded dictionary cleanly into the application.
Malformed output	Application rejects it safely; no downstream use as valid data.	Caught JSONDecodeError instantly when evaluating truncated JSON brackets.	Pass	Prevented corrupt data from reaching backend databases.
Missing field	Validation fails and defined handling occurs.	Flagged missing mandatory keys ('request_type', 'approved') during validation.	Pass	Triggered default error handling flow.
Wrong type/value	Validation catches the mismatch.	Caught type mismatch when 'days' was passed as a string ('five') instead of an int.	Pass	Type assertion successfully isolated the error.
Extra surrounding text	Parser/validator behavior is explicit and explainable.	Successfully stripped conversational markdown chatter and isolated core JSON.	Pass	Robust parser correctly handled noisy responses.
Insufficient information	Application represents uncertainty/clarification safely rather than inventing.	Model correctly stated it lacked internal company records rather than guessing.	Pass	Upheld epistemic humility and refusal guidelines.
Timeout/API failure	Application follows bounded retry/fallback/error behavior.	Simulated TimeoutError triggered retry logic and graceful fallback.	Pass	Handled network drop smoothly without breaking user experience.

Thamill Indian B U
Cohort 15

Part 3 - Engineering Decisions & Trade-offs

1. What output contract did you choose, and why? We went with a clean dictionary schema covering employee name, request type, number of days, and approval status because it maps directly to our backend database and API requirements.

2. What is one thing your validation catches that parsing alone does not? Data types and required fields. Basic parsing only checks if JSON is syntactically valid, whereas validation checks that values are of the correct type (like integers instead of strings) and that mandatory keys aren't missing.

3. Which failure did you choose to retry, if any? Why? We retry transient infrastructure issues like connection timeouts or API rate limits, since these are temporary network glitches unrelated to model intelligence.

4. What happens when retry is exhausted or not appropriate? We log the failure, stop retrying, and show a clean fallback message or route to manual review so the system doesn't hang indefinitely.

5. What is one trade-off introduced by stricter validation or fallback behavior? Slightly higher latency from retry backoffs, extra token overhead for prompt formatting, and occasional false negatives where valid answers get blocked by overly rigid rules.

6. Which reliability control belongs outside the prompt? Why? Schema validation and type checks must happen in standard application code outside the prompt. Prompts guide the model, but deterministic code ensures actual system safety.

Part 4 - Analyze Failure Behavior

7. Which failure case was most important or surprising? Malformed outputs and type mismatches. It's surprising how often an LLM can return broken syntax or unexpected string types, which will instantly crash backend databases if unhandled.

8. Can your application receive a valid-looking response that is still wrong? Explain. Yes. An LLM can generate perfect, syntactically valid JSON with correct data types, but still contain factual hallucinations or logical nonsense (like approving 500 days of leave).

9. What failure could still reach the user even after your current controls? Semantic hallucinations and factual errors that pass basic structural checks. Code verifies syntax and types, but it can't verify real-world business logic without database lookups.

10. What would you test next if this were moving toward production? Integrating Retrieval-Augmented Generation (RAG) with vector databases and adding automated unit tests for edge cases.

Part 5 - FDE Reflection

11. If this is integrated into an employee workflow, which failures must block the workflow? Malformed outputs, missing mandatory fields, type mismatches, and policy violations that impact payroll or HR records.

12. What should the employee see when the model/service is unavailable? A clear, friendly message: 'The assistant is currently busy or offline. Please try again in a moment or contact HR.

13. What response-time or availability expectation would you clarify with the customer? Target response times under 3 seconds and 99.5% uptime with graceful degradation during outages.

14. When would human review or approval be required? Whenever high-impact financial transactions, policy exceptions, or sensitive personal data changes are involve.

15. What evidence would you show the customer before asking them to trust the integration? Automated test run logs across our reliability test cases, benchmark accuracy scores, and proof of graceful fallback during simulated outages.

Part 6 - One Improvement

One Improvement: If we had another 30 minutes, we would integrate Pydantic models for automated runtime type coercion and schema validation. This would replace manual key checking with robust object validation, automatic type casting, and clean error messages, strengthening our entire reliability pipeline.